

Acceptance — pipeline fix (P1-P13)

What this document is (and is not)

This is a **before/after acceptance note** for the generation-pipeline fixes. It shows how the pipeline's *rules* changed and demonstrates the *real, offline, pure decision logic* that now guards the pipeline.

Honesty / scope — read this first:

- The synopsis and script excerpts below are **fixtures** — small hand-written examples chosen to illustrate the difference the prompt-rule changes make. They are **not** the output of a paid LLM run. No paid text or video generation was performed for this document (that is out of scope without paid API calls).
 - Every block explicitly labelled **“REAL pure-function run”** is the *actual, verbatim* stdout of the shipped pure functions (`lib/scene-breakdown.ts` , `lib/prompts/shot.ts`) executed on the fixtures. These functions import nothing at runtime (no DB, no network, no LLM), so their output is fully reproducible.
 - Static checks, pure-function runs and prompt-text inspection are **not** the same as watching a finished rendered video. Nothing here claims a video was rendered or watched.
 - Nothing here claims the show is now “guaranteed interesting” or commercially successful. The fixes remove specific *mechanical* failure modes (lost events, invented dialogue, forced aggression, silent truncation, stale derived data, subtitle artifacts, clip-length clamping, dropped finales). Whether the resulting story is *good* is a creative judgement this document does not assert.
-

1. Before / after synopsis (FIXTURE — illustrates prompt-rule differences)

Fixture only. Not a paid generation. Same episode premise, shown under the OLD rule-set vs the NEW one.

BEFORE (old rules — over-forced drama, description-driven):

Ep. 4 — “The Ledger.” Mara discovers Victor forged the accounts. The episode description says it is a tense confrontation, so the breakdown **invents extra beats to hit a drama quota**: a shouting match, a shoved bookshelf, a slap, and a dramatic keyProp (a shattered whisky glass) is **added even though the script never mentions one**. Victor, who in the script simply refuses to explain, is **rewritten into a physical aggressor** because the old rule mandated escalation to a fight. A “final” scene the model produced past the ceiling is **silently sliced off**, so the episode ends on the argument, not the intended cold departure.

AFTER (new rules — script is the single source of truth):

Ep. 4 — “The Ledger.” The breakdown is built from the **approved script**, not the description. Mara lays the forged ledger on the desk (the event). She accuses him once. Victor **refuses — silently** (the mute reaction is preserved; no dialogue is invented for him). Mara leaves; the desk lamp dies (the intended cold cliffhanger — the finale is **kept**, never sliced). **No keyProp is invented, no fight is forced**, and going over the scene ceiling is **reported explicitly** instead of silently trimming the ending.

2. Before / after script of one episode (FIXTURE — same labeling)

Fixture only. One scene of Ep. 4, showing the same premise before/after the rule change.

BEFORE (old rules):

```
INT. PENTHOUSE STUDY — NIGHT
MARA:   You signed this!
VICTOR: (invented) You don't understand what they'd have done!
        -- they struggle; VICTOR sweeps a whisky glass off the desk; it SHATTERS. (in-
        vented keyProp)
VICTOR: (invented) I did it for us!
        -- MARA slaps him. (forced physical aggression)
[SCENE CONTINUES — an extra argument beat added to hit the drama quota]
[final "cold departure" scene produced by the model is DROPPED by slice(0,
MAX_SCENES)]
```

AFTER (new rules):

```

INT. PENTHOUSE STUDY – NIGHT
MARA lays the forged ledger on the desk.
MARA: You signed this.
VICTOR turns to the window. He says nothing.           (mute reaction – preserved, not
invented)
MARA sets the ledger down and leaves.
MARA: Then we're done.
The desk lamp flickers out.                             (final/cliffhanger scene –
kept, not sliced)

```

3. Scene-breakdown of the fixed script — REAL pure-function run

The following blocks are the **actual verbatim stdout** of `validateSceneCoverage`, `scriptApprovalState`, `scriptFingerprint`, `pickPredecessorState` and `dependentStateIds` from `lib/scene-breakdown.ts`, run on the fixtures above. (Reproduce by calling these exported pure functions on the same fixtures.)

3a. Approval gate — the breakdown only runs from an APPROVED script (`scriptApprovalState`)

```

BEFORE (empty script) -> {
  "approved": false,
  "reason": "This episode has no approved script yet. Generate and approve the episode
script first – the scene breakdown is built from the approved script, not from the
synopsis/description."
}
AFTER (approved script) -> {
  "approved": true,
  "reason": "Approved script present."
}

```

3b. Stale-detection — derived scenes are invalidated when the script's *content* changes (`scriptFingerprint`)

```

fp(v1)                = 3f751b9-67
fp(v1 reformatted)    = 3f751b9-67    (whitespace-only edit -> same hash; cosmetic re-
format is NOT a change)
fp(v2 real edit)      = 39ea0cbd-83    (content edit -> different hash => derived scenes
are STALE)

```

3c. Coverage of the fixed (AFTER) script — no problems (`validateSceneCoverage`)

The AFTER breakdown includes an **action-only scene** (Victor's mute refusal, empty dialogue). It is **not** flagged as "missing speaker" — a scene with no dialogue is legal, so the mute reaction survives.

```
{
  "problems": [],
  "overLimit": false,
  "count": 3,
  "maxScenes": 12,
  "overflow": 0,
  "missingAction": [],
  "missingSpeakers": []
}
```

3d. The OLD failure modes are now REPORTED, not silently trimmed (validateSceneCoverage)

A breakdown that loses a scene's event, or has dialogue with no SPEAKER label, is caught:

```
{
  "problems": [
    "scene 2 has no action – the scene's event is missing",
    "scene 3 dialogue has no SPEAKER-labelled line – speakers must be preserved from the script"
  ],
  "overLimit": false,
  "count": 3,
  "maxScenes": 12,
  "overflow": 0,
  "missingAction": [ 2 ],
  "missingSpeakers": [ 3 ]
}
```

3e. Over the production ceiling is EXPLICIT — the finale is never silently sliced (validateSceneCoverage)

The old worker did `scenes.slice(0, MAX_SCENES)`, silently dropping any scene past the ceiling (including the finale). Now overflow is reported so the caller trims with a logged note, ceiling-aware, keeping the closing scene:

```
{
  "problems": [],
  "overLimit": true,
  "count": 6,
  "maxScenes": 4,
  "overflow": 2,
  "missingAction": [],
  "missingSpeakers": []
}
```

3f. Reworking an early episode uses the correct predecessor and marks dependents stale

Rework episode 2. The predecessor must be episode 1's state (largest `reflectsEpisodeNumber` **below** 2) —

not the newest row by `updatedAt` (episode 3's state, which is *later* in the story). Episode 3's state becomes a dependent to mark stale:

```
predecessor for episode 2 -> {
  "id": "s1",
  "reflectsEpisodeNumber": 1,
  "updatedAt": "2024-01-02"
}
dependents of episode 2 (to mark stale) -> [ "s3" ]
```

4. Sequential shot-plan / video-prompt fragment — REAL pure-function run

This is the **actual verbatim output** of `assembleShotPrompt ( lib/prompts/shot.ts )` for a **sequence** of

four shots of the AFTER scene. It demonstrates the causal chain **event → line → reaction → consequence**

across consecutive shots (not a single isolated “good” shot), carried by the `MATCH-CUT IN/OUT` continuity

blocks:

SHOT 1 — EVENT (Mara lays the ledger down)

STYLE: gritty neo-noir, teal-orange grade

LOCATION: penthouse study – low key, single desk lamp.

CHARACTERS (only these people are in frame):

- Mara – look: sharp bob, charcoal suit; wardrobe: charcoal suit.
- Victor – look: grey stubble; wardrobe: open-collar shirt.

START STATE (first shot of the scene – establish it fully): Victor at the window; Mara enters holding a ledger.

ACTION (one micro-action, escalation step "reveal"): a single continuous beat, no internal montage cuts.

CAMERA: over-the-shoulder / medium-close, eye-level, holding on the speaker; framing scale MCU (technique: ots-medium).

MATCH-CUT OUT (carry to the next shot): the ledger meets the desk

SHOT 2 — LINE (Mara accuses; picks up the exact instant from shot 1)

MATCH-CUT IN (continue the SAME instant from the previous shot): the ledger meets the desk

ACTION (one micro-action, escalation step "verbal"): a single continuous beat, no internal montage cuts.

LINE (spoken in English, voiced verbatim): You signed this.

MATCH-CUT OUT (carry to the next shot): Victor's jaw sets

SHOT 3 — REACTION (Victor refuses — MUTE; no invented line)

MATCH-CUT IN (continue the SAME instant from the previous shot): Victor's jaw sets

ACTION (one micro-action, escalation step "withheld"): a single continuous beat, no internal montage cuts.

MATCH-CUT OUT (carry to the next shot): he turns fully to the window

*Note: shot 3 has **no LINE block at all** — the reaction is silent, and the prompt does not fabricate a line for Victor. This is the mute-reaction case, preserved end-to-end.*

SHOT 4 — CONSEQUENCE (Mara leaves; the lamp dies — cliffhanger close)

MATCH-CUT IN (continue the SAME instant from the previous shot): he turns fully to the window

ACTION (one micro-action, escalation step "turn"): a single continuous beat, no internal montage cuts.

LINE (spoken in English, voiced verbatim): Then we're done.

END STATE (last shot of the scene — end exactly here): Mara gone; Victor alone in the dark study.

(STYLE / LOCATION / CHARACTERS / CAMERA / NEGATIVE blocks repeat on every shot; trimmed here for readability on shots 2-4. The full verbatim output is reproducible by running `assembleShotPrompt` on the same inputs.)

Non-English spoken line survives to the video prompt (REAL run)

When the spoken language is not English, the line is kept in that language and the model is told to voice

the English translation verbatim — the language is not silently anglicised away:

LINE (spoken in Russian; the model reads this English translation verbatim): You signed this.

5. What became clearer, and why

- **The script is the single source of truth.** The breakdown is gated on an *approved* script (`scriptApprovalState`) and built from it, not re-derived from the synopsis/description. The synopsis is passed as context only. → Fewer contradictions between what the script says and what gets shot.
- **Excess/invented events removed.** The old drama-quota rules that manufactured a keyProp, a shoving match and a slap are gone. A scene *need not* use a keyProp; a conflict *may* resolve by refusal, one-sided

pressure, withheld information or loaded silence — no prompt mandates a fight or physical aggression.

- **Motives are clearer because reactions are honest.** A character’s mute refusal is preserved as an action-only beat with **no invented dialogue** (coverage does not flag it; the shot prompt emits no LINE block). The reaction reads as a *choice*, not filler.
- **Causality is preserved shot-to-shot.** Each shot carries `MATCH-CUT IN/OUT`, so the sequence reads as event → line → reaction → consequence rather than a bag of disconnected “cool” shots.
- **The ending is preserved.** Going over the scene ceiling is reported (`overLimit / overflow`) instead of `slice(0, MAX_SCENES)` silently dropping the finale. The closing/cliffhanger scene is validated as present.
- **Derived data can’t silently go stale.** A content change to the script changes its fingerprint, so scenes/shots derived from an older version are marked stale rather than treated as fresh. Reworking an early episode picks the correct predecessor state (by `reflectsEpisodeNumber`, not `updatedAt`) and marks the later dependent states stale.
- **No subtitle artifacts, and assembly trusts the real footage.** Subtitles were removed from the product and pipeline. Assembly uses the **actual** probed clip duration; a provider clip longer than the plan is used in full rather than clamped to the planned length, and a line is never silently truncated.

6. What actually ran vs what is illustration

Item	Status
<code>scriptApprovalState</code> , <code>scriptFingerprint</code> , <code>validateSceneCoverage</code> , <code>pickPredecessorState</code> , <code>dependentStateIds</code> output (§3)	REAL pure-function run on fixtures (verbatim stdout)
<code>assembleShotPrompt</code> sequence + non-English line (§4)	REAL pure-function run on fixtures (verbatim stdout)
Before/after synopsis (§1) and script (§2)	FIXTURE — hand-written to illustrate prompt-rule differences; not a paid generation
Rendered/watched video	NOT performed — out of scope without paid generation; no such claim is made
“The show is now interesting / commercially successful”	NOT claimed

Automated coverage of the 12 mandatory cases

The 12 mandatory pipeline cases are covered by offline tests and asserted by a single unified runner:

```
npx tsx --tsconfig tsconfig.json scripts/test-stage195-all-12.ts
```

It runs each backing test suite and prints, per case, the covering test and PASS/FAIL, ending with the total (12/12). `scripts/test-stage194.ts` adds explicit asserts for the previously thin cases (mute reaction, no invented keyProp, refusal $\neq$ forced aggression, non-English line survival, no silent truncation). All runs are pure/offline — no DB, no network, no LLM, no paid generation.